

Module 1 Portfolio Project: TicketFlowDB

A guided SQL Server portfolio build

Project goal

Build a fictional event-ticket management database in SQL Server. This is a new database and does not modify TechHealthDB.

The project matches Module 1 of **Data Manipulation and Transactions in SQL Server**:

- Introduction to SQL Server
- Basic T-SQL syntax and commands
- Creating a database
- Creating related tables
- Inserting data
- Retrieving data with SELECT
- Modifying data with UPDATE
- Removing data with DELETE
- Using precise WHERE clauses

Transactions, TRY . . . CATCH, ACID properties, and isolation levels are intentionally excluded because they belong to later modules.

Business scenario

TicketFlow is a fictional company that manages customers, events, venues, and ticket bookings. Staff need a database to:

1. Register customers.
2. Create venues and events.
3. Record ticket bookings.
4. View available and booked tickets.
5. Update customer, event, and booking information.
6. Remove cancelled or test records safely.

Database design

Customers

Column	Suggested type	Rules
customer_id	INT IDENTITY(1,1)	Primary key
full_name	VARCHAR(100)	Required
email	VARCHAR(150)	Required and unique
city	VARCHAR(80)	Required
registration_date	DATE	Default to current date

Venues

Column	Suggested type	Rules
venue_id	INT IDENTITY(1,1)	Primary key
venue_name	VARCHAR(120)	Required
city	VARCHAR(80)	Required
capacity	INT	Must be greater than zero

Events

Column	Suggested type	Rules
event_id	INT IDENTITY(1,1)	Primary key
venue_id	INT	Foreign key to Venues
event_name	VARCHAR(150)	Required
event_date	DATE	Required
category	VARCHAR(50)	Required
ticket_price	DECIMAL(10,2)	Must be greater than zero
tickets_available	INT	Must not be negative
event_status	VARCHAR(20)	Scheduled, Cancelled, or Completed

Bookings

Column	Suggested type	Rules
booking_id	INT IDENTITY(1,1)	Primary key
customer_id	INT	Foreign key to Customers
event_id	INT	Foreign key to Events
booking_date	DATE	Default to current date
quantity	INT	Must be greater than zero
total_amount	DECIMAL(10,2)	Must be greater than zero
booking_status	VARCHAR(20)	Confirmed or Cancelled

Stage 1: Create the database

Create and select the new database:

```
CREATE DATABASE TicketFlowDB;
GO

USE TicketFlowDB;
GO
```

Verification:

```
SELECT DB_NAME() AS CurrentDatabase;
```

Expected result: TicketFlowDB.

Stage 2: Create the tables

Create the tables in this order:

1. Customers
2. Venues
3. Events
4. Bookings

Add:

- Primary keys
- Foreign keys
- NOT NULL rules
- Appropriate DEFAULT values
- UNIQUE constraint for customer email
- CHECK constraints for prices, quantities, capacity, available tickets, and statuses

Verification:

```
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
```

```
WHERE TABLE_TYPE = 'BASE TABLE'  
ORDER BY TABLE_NAME;
```

Expected result: four tables.

Stage 3: Insert sample data

Insert at least:

- 8 customers from different cities
- 3 venues
- 6 events in different categories
- 10 bookings

Use both single-row and multi-row INSERT statements.

Example:

```
INSERT INTO Customers (full_name, email, city)  
VALUES ('Maya Patel', 'maya.patel@example.com', 'London');
```

Verification:

```
SELECT COUNT(*) AS CustomerCount FROM Customers;  
SELECT COUNT(*) AS VenueCount FROM Venues;  
SELECT COUNT(*) AS EventCount FROM Events;  
SELECT COUNT(*) AS BookingCount FROM Bookings;
```

Stage 4: Retrieve data with SELECT

Write and save queries for:

1. All customers
2. All scheduled events
3. Events costing more than £40
4. Events ordered from cheapest to most expensive
5. Customers from one selected city
6. Confirmed bookings
7. Bookings containing two or more tickets
8. Events occurring within a chosen date range
9. The five most expensive events using TOP
10. Distinct event categories using DISTINCT

Practise:

- Column aliases
- WHERE
- AND and OR

- IN
- BETWEEN
- LIKE
- ORDER BY
- TOP
- DISTINCT

Stage 5: Modify data with UPDATE

Complete these business changes:

1. Correct one customer's city.
2. Change one customer's email address.
3. Increase the price of one event.
4. Mark a past event as Completed.
5. Reduce available tickets for a chosen event.
6. Mark one booking as Cancelled.

Always preview the target rows:

```
SELECT *
FROM Events
WHERE event_id = 1;
```

Then run a targeted update:

```
UPDATE Events
SET ticket_price = 55.00
WHERE event_id = 1;
```

Run the same SELECT again to prove the change.

Stage 6: Remove data with DELETE

Create a clearly labelled test customer who has no bookings. Then:

1. Preview the exact customer.
2. Delete only that customer.
3. Verify that the customer no longer exists.

```
SELECT *
FROM Customers
WHERE email = 'delete.test@example.com';

DELETE FROM Customers
WHERE email = 'delete.test@example.com';

SELECT *
FROM Customers
WHERE email = 'delete.test@example.com';
```

The final query should return zero rows.

Also demonstrate that SQL Server protects related data by attempting to delete a customer who has a booking. Record the foreign-key error in your project notes; do not remove the constraint.

Stage 7: Final business questions

Write queries that answer:

1. Which events are currently scheduled?
2. Which events have fewer than 50 tickets available?
3. Which customers registered most recently?
4. Which bookings have been cancelled?
5. Which events belong to the Music or Technology categories?
6. Which customers have names beginning with A or M?
7. Which events cost between £25 and £75?
8. Which bookings have a total value above £100?

For each query, add a comment explaining the business question.

Safety checklist for DML

Before every UPDATE or DELETE:

1. Write the WHERE condition.
2. Test it with SELECT.
3. Confirm the returned rows are exactly the intended records.
4. Run the UPDATE or DELETE.
5. Run a verification SELECT.

Never practise an unrestricted UPDATE or DELETE on your portfolio data.

Portfolio folder structure

```
TicketFlowDB/
  README.md
  sql/
    01_create_database.sql
    02_create_tables.sql
    03_insert_sample_data.sql
    04_select_queries.sql
    05_update_queries.sql
    06_delete_queries.sql
    07_business_questions.sql
  diagrams/
    ticketflow-erd.png
  evidence/
    database-and-tables.png
    inserted-data.png
    select-results.png
    update-before-after.png
    delete-before-after.png
```

README contents

Include:

- Project title and overview
- Fictional business problem
- Technologies used: SQL Server, T-SQL, Docker, and VS Code
- Table descriptions and relationships
- Skills demonstrated
- Instructions for running scripts in numerical order
- Examples of INSERT, SELECT, UPDATE, and DELETE
- Screenshots showing your results
- What you learned from Module 1

Completion checklist

- ☐ Created TicketFlowDB
- ☐ Created four related tables
- ☐ Added keys and data-integrity constraints
- ☐ Inserted realistic fictional sample data
- ☐ Completed all retrieval exercises
- ☐ Demonstrated targeted updates with before-and-after evidence
- ☐ Demonstrated a safe deletion
- ☐ Demonstrated foreign-key protection
- ☐ Answered all eight business questions
- ☐ Created an ER diagram
- ☐ Added scripts, screenshots, and documentation to the portfolio

Portfolio statement

Designed and built a relational SQL Server database for a fictional event-ticketing business. Created related tables with primary keys, foreign keys, and validation constraints, populated realistic sample data, and wrote targeted T-SQL INSERT, SELECT, UPDATE, and DELETE operations to answer business questions and maintain data integrity.